Angel Castillo Serrato

D M A

Scribe

Preguntas Abiertas 8

1.- ¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

La abstracción consiste en identificar y conservar únicamente las características y comportamientos que son relevantes y útiles para resolver el problema específico en un contexto determinado, descartando lo irrelevante.

En un contexto de diseñar una televisión cuyo propósito es cambiar de canal, se conserva el atributo de número de canal y omitimos la marca.

2.- ¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

El constructor sirve para inicializar el estado de un objeto o instancia al momento de ser creado en memoria mediante el operador new. Se distingue de un método común en que debe llevar obligatoriamente el mismo nombre que la clase y no declara ningún tipo de retorno.

3.- ¿Qué implica final en una variable? Si guarda una referencia a un arreglo, ¿impide cambiar sus elementos?

La palabra reservada final impide que una variable sea reasignada a una nueva referencia o a un nuevo valor una vez que ha sido inicializada.

Si la variable guarda una referencia a un arreglo, final impide cambiar la referencia para apuntar a un arreglo diferente en memoria, pero no impide modificar lo de dentro.

4.- ¿Por qué == no basta para comparar el contenido de dos String? Explica qué usarías para comparar sus textos.

El operador == compara la identidad de las referencias en memoria, no la secuencia de caracteres del texto. Para comparar el contenido o texto de 2 objetos

String se debe utilizar el método .equals(), el cual evalúa caracter por caracter el contenido textual

6.- ¿En qué se distinguen while y while? Explica qué ocurre si ya no se vuelve a ejecutar el inicio:

El ciclo while es de pre-prueba (evalúa una vez antes de ejecutar) y mientras que el do-while es como una post-prueba. Ambos tienen el inicio en el cual while puede ejecutarse 0 veces y producirse ya que se ejecuta (0 o más iteraciones). En el segundo, el cuerpo del bucle se ejecuta exactamente una vez antes de verificar la condición y finalizar.

R1. ¿QUÉ IMPRIME?

- Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

La variable fila guarda una referencia al mismo arreglo que tabla[0] (la fila). Por lo tanto, modificar fila[i] mediante la división entera altera directamente el valor almacenado en tabla[0][i].

Iteración / Linea	i	fila[i]	fila[i] = fila[i]/2	tabla[0][i]	Acumulado	Salida
Inicio			{9, 4, 7}	0		
1 (i=0)	0	9	9/2 = 4	4	0+4	0:4
2 (i=1)	1	4	4/2 = 2	2	4+2	1:6
3 (i=2)	2	7	7/2 = 3	3	6+3	2:9
Println (double)					9	4.0
Println 5 ("P"+000)				fila[0] = 4 tabla[1][0] = 2	9	P42

R2. Analisis de fragmentos de codigo

a) Arreglo y ciclo for con límite de indice

- Compila pero truena

La condición de parada del ciclo utiliza el operador <= (i <= datos.length, i <= 2). En la iteración donde i = 2, se intenta acceder a datos[2]. Dado que los índices válidos de un arreglo de tamaño 2 son únicamente 0 y 1, Java arroja la excepción ArrayIndexOutOfBoundsException en tiempo de ejecución en esa línea.

b) Unboxing de Integer y concatenación

- Funcona
- 8 / P6

En piezas + 2, Java realiza un unboxing automático del objeto wrapper Integer a su valor primitivo para realizar la suma (8 + 2 = 8). En la línea 3 ("P" + P) tipo String, Java realiza una concatenación convirtiendo el objeto a su representación textual.

c) Análisis de rutas de retorno en métodos

- No Compila

El método declara un tipo de retorno int pero el compilador de Java existe que todos los caminos posibles de ejecución retornen un valor del tipo especificado. Si la variable valor es menor o igual a 0 la condición del if es falsa y el método finaliza sin ejecutar ninguna sentencia return.

R3. MODELADO DE LÁMPARA

Enum dado: enum Estado {APAGADO, ENCENDIDO, PARPADEO}

Codigo:

```
public class Lampara {
    final String etiqueta;
    Estado estado;
    static int contadorLamparas = 0;
    public Lampara (String etiqueta){
        this.etiqueta = etiqueta;
        this.estado = Estado.APAGADO;
        contadorLamparas++;
    }
    public void encender() {
        this.estado = Estado.ENCENDIDO;
    }
    public Estado getEstado() {
        return this.estado;
    }
}
```

- Tipo de etiqueta: Es de tipo String porque almacena texto
- etiqueta: Es de instancia para que cada lampara y final porque es fija e inmutable.
- Tipo y Reto de estado: Es de Enum y nace con un valor inicial como se especifica
- estado: Es de instancia y mutable para que pueda cambiar su estado
- contadorLamparas: Es int y static para llevar la cuenta y global de las lamparas creadas.

- Constructor: Inicializa la etiqueta, establece el estado inicial en APAGADO e incrementa el contador static.
- Metodo encender: Metodo de instancia que actualiza el estado de la lampara a "Estado: ENCENDIDO".
- Metodo getEstado: Devuelve el valor actual del atributo.

B4. CODIGO CON REPETICIÓN

```
static int posicionMinima(int[] temperaturas){
    int pmtemp = 0;
    for(int i=1; i< temperaturas.length; i++){
        if(temperaturas[i] < temperaturas[pmtemp]){
            pmtemp = i;
        }
    }
    return pmtemp;
}
```

- {8, 3, 5}
  - pmtemp = 0 (8).
  - Iteración i = 1 & temperaturas[1] = 3
    Se evalua 3 < 8 (True)
    pmtemp = 1
  - Iteración i = 2 & temperaturas[2] = 5
    Se evalua 5 < 3 (falso)
    pmtemp = 1
  - Finaliza el ciclo y retorn 1.

- {7}
  - pmtemp = 0
  - Condición for i < temperatura.length (1<1) (falsa)
  - Return 0
- {-2, -8, -1}
  - pmtemp = 0 (-2)
  - iteración i = 1 & temperatura[1] = -8 (-8 < -2) ~~falso~~ verdadero pmtemp = 1
  - iteración i = 2 & temperatura[2] = -1 (-1 < -8) falso pmtemp no cambia
  - return 1

R5. CÓDIGO CON SELECCIÓN

```
Static String mantenimiento (int ciclos) {
    if (ciclos <=0) {
        return "INVALIDO";
    } else if (ciclos % 24 ==0) {
        return "COMPLETO";
    } else if (ciclos % 12 ==0) {
        return "REVISIÓN";
    } else if (ciclos % 4 ==0) {
        return "BASICO";
    } else {
        return "NINGUNO";
    }
}
```

Regla / Valor recibido	Condición	Texto retornado	Explicación
Menor o igual que 0	ciclos <= 0	"INVALIDO"	Cualquier valor ≤ 0 se intercepta aquí retorna sin evaluar más.
Divisible entre 24	ciclos == 100	"COMPLETO"	El operador residuo % devuelve 0 si es multiplo de 24. Captura números multiplos de 24.
Entre 12	ciclos % 12 == 0	"REVISIÓN"	Evalua si es divisible entre 12 si no fue entre 24
Entre 4	ciclos % 4 == 0	"BASICO"	Evalua si es multiplo del 4. Si no fue del 12 ni 24.
Cualquier otro	else	"NINGUNO"	Otro cualquiera que no sea divisible cae aquí en el else.